

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA Informatică

LUCRARE DE LICENȚĂ

**Sentinel: Sistem de Detectare și
Răspuns la Intruziuni bazat pe eBPF
cu Scoring Anomalistic Augmentat
prin Învățare Automată**

Conducător științific
[Grad, titlu și nume coordonator]

Absolvent
[Nume student]

ABSTRACT

This thesis presents the design and implementation of Sentinel, an eBPF-based Host Intrusion Detection and Response System (HIDS) for the Linux platform. Sentinel attaches kprobes to 21 security-relevant system calls, collects events through a 64 MiB ring buffer and processes them in a Go user-space daemon. A three-layer hybrid scoring engine assigns risk scores: static per-syscall weights calibrated on empirical malware data, a sliding-window burst detector, and an optional Isolation Forest ML scorer served through a REST API. Scores are mapped to six severity states (OK / WATCH / WARN / THROTTLE / ISOLATE / FREEZE) and a quarantine manager applies proportional containment actions via Linux cgroup v2 and nftables — CPU throttling at 10%, memory capping at 256 MiB, network isolation by cgroup inode, and full process freezing via SIGSTOP and `cgroup.freeze`. The system is validated against a reproducible suite of 21 attack scenarios covering kernel rootkits, privilege escalation, process injection, network exfiltration, persistence and fileless execution. Measured containment latency is below 5 ms; eBPF overhead is approximately 1.2% CPU on a kernel-build workload. Original contributions include the adaptive weighted scoring mechanism, the cgroup-inode-based selective network isolation, the PID-reuse-safe burst key, and the self-contained attack test suite.

Cuprins

1	Introducere	1
1.1	Contextul și motivația	1
1.2	Enunțul problemei	2
1.3	Obiectivele lucrării	2
1.4	Contribuții originale	3
1.5	Structura lucrării	3
2	Fundamente teoretice și contextul actual	5
2.1	Modelul de securitate Linux	5
2.1.1	Apeluri de sistem și frontiera kernel-utilizator	5
2.1.2	Linux Security Modules	6
2.1.3	Namespaces și grupuri de control (cgroups)	6
2.2	Extended Berkeley Packet Filter (eBPF)	7
2.2.1	Evoluție de la BPF clasic la eBPF	7
2.2.2	Maps eBPF și mecanismul ring buffer	7
2.2.3	Kprobes: instrumentarea dinamică a kernel-ului	8
2.3	Sisteme de detectare a intruziunilor	8
2.3.1	Taxonomie și clasificare	8
2.3.2	Detecție bazată pe anomalii versus semnături	9
2.4	Algoritmul Isolation Forest	9
2.5	Instrumente existente și lucrări corelate	10
3	Analiza și Proiectarea Sistemului Sentinel	12
3.1	Cerințe funcționale și nefuncționale	12
3.2	Arhitectura sistemului	13
3.3	Proiectarea programului eBPF	13
3.4	Proiectarea motorului de scoring	14
3.4.1	Stratul 1: Ponderi statice per syscall	15
3.4.2	Stratul 2: Detectarea activității în rafale	15
3.4.3	Stratul 3: Alerte bazate pe căi de fișiere	16
3.4.4	Integrarea scorerului ML	16

3.5	Mașina de stări pentru carantinare	17
3.6	Schema bazei de date	17
4	Implementare și Testare	19
4.1	Mediul de dezvoltare și testare	19
4.2	Implementarea programului eBPF	19
4.3	Implementarea daemonului Go	20
4.4	Implementarea scorerului ML	21
4.5	Implementarea managerului de carantinare	22
4.6	Suita de teste de atac	23
4.6.1	Metodologia de testare	23
4.6.2	Scenariile de bază (REP-01 – REP-12)	24
4.6.3	Scenariile avansate (REP-13 – REP-21)	24
4.7	Considerente de performanță	25
5	Concluzii	27
	Contribuții originale	27
	Limitări	28
	Direcții de cercetare viitoare	28
	Bibliografie	30

Capitolul 1

Introducere

Securitatea sistemelor de operare bazate pe Linux a dobândit o importanță tot mai mare pe măsură ce aceste sisteme au ajuns să susțină infrastructuri critice: servere web, platforme de containere, sisteme de calcul de înaltă performanță și dispozitive IoT. Creșterea numărului de atacuri direcționate împotriva nucleului Linux — rootkit-uri, tehnici de injecție de procese, escaladare de privilegii și execuție fără fișier pe disc — impune soluții de detecție capabile să opereze la nivelul cel mai profund al stivei software, fără a introduce latențe inacceptabile sau vulnerabilități suplimentare.

Lucrarea de față descrie proiectarea și implementarea sistemului *Sentinel*, un sistem de detectare și răspuns la intruziuni bazat pe tehnologia *extended Berkeley Packet Filter* (eBPF). Sistemul interceptează apeluri de sistem la nivel de kernel prin mecanismul *kprobe*, calculează un scor de risc compus pentru fiecare eveniment detectat și aplică automat acțiuni graduale de containment al proceselor suspectate. *Sentinel* funcționează fără a modifica codul kernel-ului și fără a fi necesară repornirea sistemului — proprietăți esențiale pentru medii de producție.

1.1 Contextul și motivația

Instrumentele tradiționale de monitorizare a securității, cum ar fi demonul de audit Linux *auditd* [Lin24] sau sistemul Snort bazat pe semnături de rețea [Roe99], au demonstrat limitări documentate în fața atacurilor sofisticate care operează exclusiv în memorie sau care manipulează structurile interne ale kernel-ului [CGFB18]. Abordările bazate pe analiza jurnalelor suferă de o latență inerentă: evenimentul este produs, jurnalizat, scris pe disc și abia apoi analizat, permițând atacatorilor o fereastră de acțiune neobservată de ordinul secundelor sau minutelor.

eBPF oferă o paradigmă diferită. Programele eBPF se execută într-un sandbox verificat formal al kernel-ului Linux, atașate la puncte de instrumentare (*kprobes*,

tracepoints, uprobes) cu o suprasarcină de ordinul zecilor de nanosecunde per eveniment [Gre19]. Aceasta permite vizibilitate completă, în timp real, asupra comportamentului oricărui proces, fără modificarea kernel-ului sau instalarea de module de kernel suplimentare.

Complementar, metodele de detecție a anomaliilor bazate pe învățare automată — în special algoritmul Isolation Forest [LTZ08] — s-au dovedit eficiente pentru identificarea comportamentelor atipice în spații de caracteristici de dimensionalitate medie, inclusiv în contextul securității informatice [BG16]. Integrarea unui scorer ML cu un agent de detecție bazat pe eBPF formează o arhitectură hibridă care combină viteza și determinismul regulilor statice cu capacitatea de generalizare a modelelor statistice.

1.2 Enunțul problemei

Problema adresată de această lucrare poate fi formulată astfel: *cum poate fi proiectat un sistem de detectare și răspuns la intruziuni pentru Linux care (a) operează la nivel de kernel fără a-l modifica, (b) clasifică evenimentele de securitate în timp real pe baza unui scor de risc compus și (c) aplică automat acțiuni de containment proporționale cu severitatea detectată, cu o latență de răspuns sub cinci milisecunde?*

Validarea empirică se realizează printr-o suită de 21 de scenarii de atac reproductibile, ce acoperă principalele categorii de amenințări documentate în literatura de specialitate: rootkit-uri de kernel, escaladare de privilegii, injecție de procese, exfiltrare de date în rețea, persistență și execuție de cod fără fișier pe disc.

1.3 Obiectivele lucrării

Obiectivele principale ale lucrării sunt:

- Proiectarea și implementarea unui program eBPF pentru interceptia selectivă a apelurilor de sistem relevante din perspectiva securității, cu suprasarcină minimă asupra sistemului.
- Construirea unui motor de scoring hibrid care combină ponderi statice per syscall, detecție a activității în rafale (burst detection) și un scorer ML bazat pe Isolation Forest, care poate fi antrenat cu date actualizate fără întreruperi.
- Implementarea unui agent de carantinare care aplică acțiuni de izolare graduale — limitare CPU, izolare de rețea, înghețarea procesului — prin mecanismele cgroup v2 și nftables.

- Validarea întregului sistem printr-o suită de teste automatizate ce rulează 21 de scenarii de atac într-un mediu izolat bazat pe mașini virtuale Vagrant.

1.4 Contribuții originale

Contribuțiile originale ale lucrării sunt:

1. **Un motor de scoring cu trei straturi:** ponderi statice ponderate per syscall, detectare a activității în rafale cu fereastră glisantă și scorer Isolation Forest care poate fi antrenat cu date actualizate prin API REST, cu un mecanism de combinare ponderată a scorurilor care previne degradarea excesivă a scorului bazal.
2. **O mașină de stări cu șase nivele** (OK / WATCH / WARN / THROTTLE / ISOLATE / FREEZE) care decuplează detecția de răspuns, permițând acțiuni proporționale cu severitatea detectată și auto-dezghețarea proceselor înghețate după un interval configurabil.
3. **O suită de 21 de scenarii de atac reproductibile**, distribuite ca artefacte de testare, care acoperă vectori de atac documentați în literatura de specialitate — de la rootkit-uri tip Diamorphine până la tehnici de execuție de cod fără fișier și rootkit-uri eBPF de tip TripleCross.
4. **Mecanismul de izolare de rețea bazat pe cgroup inode:** procesele carantinate sunt blocate în rețea prin reguli nftables care filtrează traficul după inodeul cgroup-ului, fără a afecta restul proceselor de pe sistem.

1.5 Structura lucrării

Capitolul 2 prezintă fundamentele teoretice: modelul de securitate Linux (apeluri de sistem, Linux Security Modules, namespaces și cgroups), arhitectura eBPF, clasificarea și descrierea sistemelor de detectare a intruziunilor, bazele algoritmului Isolation Forest și o analiză comparativă a instrumentelor existente. Capitolul 3 descrie analiza cerințelor și proiectarea detaliată a componentelor sistemului, inclusiv motorul de scoring și mașina de stări pentru carantinare. Capitolul 4 acoperă implementarea concretă și rezultatele testării cu suita de 21 de atacuri. Capitolul 5 sintetizează concluziile și direcțiile de cercetare viitoare.

Notă privind utilizarea instrumentelor de inteligență artificială: la redactarea unor secțiuni ale acestei lucrări a fost utilizat asistentul Claude (Anthropic) pentru reformularea și îmbunătățirea lizibilității paragrafelor tehnice. De asemenea, asistentul Claude (Anthropic) a mai fost folosit în formatarea codului și în implementarea

unor secțiuni de cod generice. Întregul conținut tehnic, implementarea software, excluzând secțiunile menționate anterior și rezultatele experimentale aparțin exclusiv autorului lucrării.

Capitolul 2

Fundamente teoretice și contextul actual

Capitolul de față oferă bazele teoretice necesare înțelegerii arhitecturii sistemului Sentinel. Sunt prezentate mecanismele de securitate ale nucleului Linux, tehnologia eBPF, clasificarea sistemelor de detectare a intruziunilor, principiile algoritmului Isolation Forest și o analiză comparativă a instrumentelor existente în domeniu.

2.1 Modelul de securitate Linux

2.1.1 Apeluri de sistem și frontiera kernel-utilizator

Frontiera fundamentală între codul cu privilegii reduse (spațiul utilizator) și codul privilegiat (kernel) în Linux este interfața apelurilor de sistem (syscall). Fiecare operație care implică acces la resurse hardware — creare de socket, execuție de proces, manipulare de fișiere, modificare de capabilități — traversează această interfață printr-o tranziție controlată [Ker10]. Din perspectiva securității, monitorizarea apelurilor de sistem oferă o vizibilitate completă și care nu poate fi evitată asupra comportamentului unui proces: un proces care instalează un modul de kernel va apela `finit_module` (syscall 313), cel care încearcă escaladarea de privilegii va apela `setuid` (syscall 105) sau `capset` (syscall 126), iar cel care exfiltrează date va genera apeluri `socket` și `connect` (syscall-urile 41 și 42) [Lov10].

Kernel-ul Linux x86.64 definește la momentul scrierii acestei lucrări peste 300 de apeluri de sistem, dar numai o fracție dintre acestea prezintă relevanță din perspectiva securității. Sistemul Sentinel monitorizează 21 de apeluri de sistem selectate pe baza frecvenței de apariție în malware-ul Linux documentat empiric [CGFB18], acoperind categoriile: execuție de procese, modificare de privilegii, operații de memorie, activitate de rețea, operații pe fișiere sensibile și management de module de kernel. Tabelul 2.1 prezintă o selecție cu greutățile de risc corespunzătoare.

Syscall	Nr.	Categorie alertă	Pondere
fininit_module	313	HiddenPID	95
init_module	175	HiddenPID	95
delete_module	176	HiddenPID	85
ptrace	101	ProcessInjection	75
setuid	105	PrivilegeEscalation	70
setgid	106	PrivilegeEscalation	70
capset	126	PrivilegeEscalation	70
execve	59	ParentChildAnomaly	50
mprotect	10	MemoryInjection	45
mmap	9	MemoryInjection	40
connect	42	NetworkAnomaly	15
socket	41	NetworkAnomaly	10

Tabela 2.1: Selecție de apeluri de sistem monitorizate de Sentinel cu greutatea de risc asociate

2.1.2 Linux Security Modules

Cadrul Linux Security Modules (LSM), descris în detaliu de Wright et al. [WCS⁺02], definește o serie de puncte de interceptare (hooks) inserate în punctele de decizie critice ale kernel-ului și permite modulelor de securitate — SELinux, AppArmor, SMACK — să impună politici de control al accesului obligatoriu (MAC). LSM operează prin politici declarative statice și nu implementează analiză comportamentală a secvențelor de apeluri de sistem; acesta este motivul pentru care un sistem de detectare bazat pe comportament dinamic rămâne complementar față de mecanismele LSM existente.

Un aspect important pentru proiectarea Sentinel este că eBPF poate fi atașat la punctele de interceptare (hook) LSM printr-un tip special de program BPF (BPF_PROG_TYPE_LSM), introdus în kernel 5.7. Sentinelul utilizează totuși kprobes, care oferă o acoperire mai largă și o portabilitate mai bună pe versiunile de kernel ≥ 5.4 .

2.1.3 Namespaces și grupuri de control (cgroups)

Namespaces Linux (PID, net, mount, user, UTS, IPC) partajează starea globală a kernel-ului în contexte izolate pentru grupuri de procese, furnizând mecanismul de bază pentru containerizare [Ker10]. Control Groups v2 (cgroup v2), propuse inițial de Menage [Men07] și standardizate complet în kernel 4.5, permit limitarea și contorizarea utilizării resurselor — CPU, memorie, I/O de bloc — la nivelul unui grup ierarhic de procese.

Cgroup v2 aduce față de predecesorul său (cgroup v1) o ierarhie unificată și un model de delegare mai coerent, eliminând ambiguitățile de configurare care apăreau

când un proces era plasat simultan în cgroup-uri din mai multe ierarhii. Sentinelul impune la pornire prezența controllerelor `cpu` și `memory` în `/sys/fs/cgroup/cgroup.controllers`. Această condiție satisfăcută implicit pe Debian 12 și Ubuntu 22.04+.

2.2 Extended Berkeley Packet Filter (eBPF)

2.2.1 Evoluție de la BPF clasic la eBPF

Berkeley Packet Filter (BPF) a fost introdus de McCanne și Jacobson în 1993 ca arhitectură eficientă pentru captura de pachete în spațiul utilizator, cu o mașină virtuală simplă de 32 de biți și un set limitat de instrucțiuni [MJ93]. Importanța conceptuală a BPF constă în mutarea filtrului în kernel — pachetele sunt evaluate înainte de copierea în spațiul utilizator, eliminând transferul de date inutile prin granița privilegiată.

Linux 3.18 (2014) a extins radical această arhitectură sub denumirea eBPF: registre de 64 de biți, un set de instrucțiuni mai bogat, structuri de date persistente (maps), un verificador formal al programelor și capacitatea de atașare la puncte de instrumentare din întreg kernel-ul [Ric23]. Verificadorul eBPF garantează că orice program încărcat va termina (absența ciclurilor infinite) și nu va accesa memorie în afara granițelor definite — proprietăți esențiale pentru siguranța unui sistem de producție. Compilarea JIT transformă bytecode-ul verificat în instrucțiuni native x86_64, eliminând overhead-ul de interpretare.

Ecosistemul eBPF a evoluat rapid: bibliotecile `libbpf` și `cilium/ebpf` [Cil24] oferă o interfață Go/C de nivel înalt pentru încărcarea și gestionarea programelor eBPF, iar CO-RE (Compile Once, Run Everywhere) rezolvă problema portabilității între versiuni de kernel prin relocări BTF.

2.2.2 Maps eBPF și mecanismul ring buffer

Maps eBPF sunt structuri de date persistente, partajate între programul din kernel și procesele din spațiul utilizator. Tipurile relevante pentru Sentinel sunt:

- `BPF_MAP_TYPE_HASH`: tabel hash pentru lista de PID-uri excluse (`pid_blacklist`).
- `BPF_MAP_TYPE_PERCPU_ARRAY`: array per-CPU pentru contorizarea evenimentelor pierdute (`drop_count`).
- `BPF_MAP_TYPE_RINGBUF`: ring buffer de 64 MiB pentru transmiterea evenimentelor către daemon.

Ring buffer-ul (introdus în kernel 5.8) oferă avantaje față de perf buffer-ul precedent: elimină o copiere suplimentară de date, suportă rezervarea atomică a spațiului urmată de scriere directă și notificarea consumatorului printr-un descriptor de fișier epollable. Gregg [Gre19] documentează că ring buffer-ul reduce latența de notificare cu 20–40% față de perf buffer în scenariile de evenimente cu frecvență ridicată.

2.2.3 Kprobes: instrumentarea dinamică a kernel-ului

Kprobes sunt un mecanism de instrumentare dinamică a kernel-ului Linux care permite atașarea de handleri la aproape orice instrucțiune din codul kernel-ului fără a-l recompila [CRKH05]. La activarea unei sonde kprobe, instrucțiunea țintă este înlocuită cu un breakpoint (INT3 pe x86), iar la declanșarea acestuia controlul este transferat handler-ului eBPF înainte de execuția instrucțiunii originale. Latența introdusă este de ordinul a 100–300 ns per apel [VCP⁺20], neglijabilă în raport cu durata unui syscall tipic (microsecunde–milisecunde).

Sentinelul atașează kprobe-uri pe funcțiile `__x64_sys_{name}` corespunzătoare celor 21 de syscall-uri monitorizate. Atașarea este dinamică: la pornire, daemonul iterează peste toate programele din colecția eBPF cu prefix `kp_` și le atașează la funcțiile kernel corespunzătoare, fără a fi necesară enumerarea lor explicită în codul Go.

2.3 Sisteme de detectare a intruziunilor

2.3.1 Taxonomie și clasificare

Anderson (1980) a definit pentru prima dată conceptul de monitorizare a amenințărilor de securitate ca analiză a jurnalelor de audit pentru identificarea comportamentelor anormale sau a utilizării neautorizate [And80]. Denning (1987) a formalizat primul model teoretic al unui IDS ca un sistem bazat pe statistici și reguli, capabil să identifice deviații de la profiluri de comportament normale [Den87].

Axelsson (2000) propune o taxonomie completă a IDS-urilor după mai multe axe [Axe00]:

- *Sursa datelor*: host-based (HIDS) versus network-based (NIDS). Sentinelul este un HIDS pur: monitorizează evenimentele unui singur host la nivelul apelurilor de sistem.
- *Metodă de detecție*: detecție bazată pe semnături (misuse detection) versus detecție bazată pe anomalii. Sentinelul combină ambele: ponderi statice per syscall (semnătură implicită) și Isolation Forest (anomalie).

- *Frecvența analizei*: detecție continuă versus periodică. Sentinelul operează în timp real, cu latența limitată de viteza ring buffer-ului.
- *Răspuns*: pasiv (alertare) versus activ (containment automat). Sentinelul implementează un răspuns activ gradual.

2.3.2 Detecție bazată pe anomalii versus semnături

Detecția bazată pe semnături identifică atacuri cunoscute prin potrivire de șabloane (pattern matching). Avantajul este rata scăzută a fals-pozitivelor pentru atacurile cunoscute; dezavantajul este incapacitatea de a detecta variante noi sau atacuri zero-day [GTDVMFV09].

Detecția bazată pe anomalii construiește un model al comportamentului normal și semnalează devierile semnificative. Poate detecta atacuri necunoscute anterior, dar rata fals-pozitivelor este tipic mai ridicată [CBK09]. Un studiu comparativ al lui Goldstein și Uchida [GU16] arată că Isolation Forest performează competitiv față de alte metode nesupervizate pe seturi de date cu proporție redusă de anomalii ($< 5\%$), ceea ce corespunde scenariului real al evenimentelor de securitate.

Buczak și Guven [BG16] realizează o revizuire exhaustivă a metodelor de ML aplicate detectării intruziunilor și concluzionează că metodele de ansamblu (ensemble methods), inclusiv Isolation Forest, oferă cel mai bun echilibru între acuratețe și eficiență computațională pentru date de securitate cu dimensionalitate variabilă.

2.4 Algoritmul Isolation Forest

Isolation Forest, propus de Liu, Ting și Zhou [LTZ08] și extins teoretic în [LTZ12], este un algoritm de detecție a anomaliilor bazat pe arbori de izolare (isolation trees). Ideea centrală este că anomaliile sunt rarer și diferite față de punctele normale, deci sunt izolate mai ușor: un arbore de decizie construit prin splits aleatorii va separa un punct anomic de restul datelor cu mai puțini pași decât ar fi necesar pentru un punct normal.

Formal, fie $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ un set de date de antrenament. Un arbore de izolare se construiește recursiv: se selectează aleatoriu un atribut $q \in \{1, \dots, d\}$ și un punct de split p uniform din intervalul $[\min_q(X), \max_q(X)]$, partiționând datele până când fiecare punct este izolat sau se atinge adâncimea maximă. Scorul de anomalie al unui punct x se calculează ca:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}} \quad (2.1)$$

unde $E[h(x)]$ este adâncimea medie de izolare pe un ansamblu de arbori, iar $c(n) =$

$2H(n-1) - \frac{2(n-1)}{n}$ este adâncimea medie de căutare eșuată într-un arbore de căutare binară cu n noduri, cu H funcția armonică [LTZ12].

Scorul variază în intervalul $(0, 1)$: valori aproape de 1 indică anomalii puternice, valori aproape de 0,5 indică puncte normale sau ambigue. Sentinelul aplică o transformare sigmoidă pentru a converti scorul brut IF (care este valoarea funcției de decizie `decision_function`, negativă pentru anomalii) într-un scor de amenințare pe scala 0–100:

$$\text{threat_score} = \frac{100}{1 + e^{-25 \cdot (-\text{raw_score})}} \quad (2.2)$$

Factorul de scalare 25 a fost ales empiric pentru a produce o separare clară între punctele normale ($\text{threat_score} < 30$) și anomalii moderate ($\text{threat_score} \in [50, 70]$), menținând în același timp sensibilitate pentru valori extreme.

Complexitatea de antrenament a Isolation Forest este $O(t \cdot \psi \cdot \log \psi)$, unde t este numărul de arbori și ψ este sub-dimensionarea (numărul de exemple per arbore). Complexitatea de inferență este $O(t \cdot \log \psi)$ per eveniment — acceptabilă pentru latența impusă de RN2 (< 50 ms per scor ML).

2.5 Instrumente existente și lucrări corelate

Falco [Clo24], proiect incubat de CNCF și derivat din Sysdig, este cel mai apropiat echivalent open-source al Sentinelului: utilizează eBPF sau un driver de kernel pentru monitorizarea syscall-urilor și permite definirea de reguli de alertă în YAML. Spre deosebire de Sentinel, Falco nu implementează un scorer ML integrat, nu aplică acțiuni automate de containment (răspunsul este delegat unor sisteme externe precum Falcosidekick) și nu menține o bază de date locală a evenimentelor pentru analiza retrospectivă.

auditd [Lin24] este componenta de audit a kernel-ului Linux, standardizată și prezentă în toate distribuțiile majore. Jurnalizează apeluri de sistem pe baza unor reguli configurate static. Latența sa este mai ridicată decât eBPF, deoarece evenimentele sunt scrise în jurnale și procesate asincron; în plus, auditd nu implementează nicio formă de răspuns automat.

Garfinkel și Rosenblum (2003) [GR03] au propus utilizarea virtualizării și a introspection de mașini virtuale (VMI) pentru detectarea intruziunilor — o abordare mai puternică din perspectiva izolării față de atacator, deoarece monitorul rulează în afara sistemului monitorizat. DRAKVUF [LMP⁺14] implementează această abordare bazată pe Xen și LibVMI, oferind o alternativă pentru medii de analiză malware. Dezavantajul este că necesită virtualizare hardware și nu este aplicabilă pe sisteme bare-metal sau în containere.

kGuard [KPK12] abordează o problemă complementară — protecția kernel-ului împotriva atacurilor de tip return-to-user — prin instrumentarea statică a codului kernel la compilare. Aceste abordări de securitate prin compilare sunt ortogonale față de Sentinel, care se concentrează pe detectarea comportamentului la runtime.

O constatare empirică importantă din literatura de specialitate provine din Cozzi et al. [CGFB18]: 70% din eșantioanele de malware Linux analizate utilizează socket-uri de rețea, 30% apelează `ptrace` sau mapează pagini cu drepturi de execuție, iar 18% utilizează module de kernel. Aceste statistici au constituit criteriul de selecție și de ponderare a syscall-urilor monitorizate de Sentinel.

Capitolul 3

Analiza și Proiectarea Sistemului Sentinel

Proiectarea sistemului Sentinel pornește de la cerința de a detecta și răspunde la amenințări în timp real, cu o arhitectură modulară care separă clar instrumentarea kernel-ului, calculul scorului de risc și aplicarea acțiunilor de containment. Cele trei componente comunică printr-un flux unidirecțional: programul eBPF produce evenimente, daemonul Go le procesează și le persistă, iar managerul de carantinare le consumă și acționează.

3.1 Cerințe funcționale și nefuncționale

Cerințele funcționale principale sunt:

- **RF1:** Interceptarea apelurilor de sistem relevante din perspectiva securității la nivel de kernel, fără modificarea kernel-ului sau instalarea de module suplimentare.
- **RF2:** Calcularea unui scor de risc compus pentru fiecare eveniment detectat, utilizând cel puțin un strat de reguli statice.
- **RF3:** Clasificarea evenimentelor în șase stări de severitate, de la absența riscului (OK) la necesitatea înghețării imediate (FREEZE).
- **RF4:** Aplicarea automată a acțiunilor de carantinare proporționale cu starea detectată, prin mecanisme standard ale kernel-ului.
- **RF5:** Persistarea evenimentelor și a acțiunilor de carantinare într-o bază de date locală, accesibilă pentru analiză retrospectivă.
- **RF6:** Expunerea unui API REST pentru integrarea unui scorer ML opțional, care poate fi antrenat cu date actualizate fără oprirea sistemului.

Cerințele nefuncționale sunt:

- **RN1:** Penalizarea de performanță asupra procesorului (CPU) introdusă de instrumentarea eBPF să fie sub 2% pe un sistem cu activitate normală.
- **RN2:** Latența de la detecție la aplicarea acțiunii de carantinare (fără scorer ML) să fie sub 5 ms.
- **RN3:** Sistemul să funcționeze pe kernel Linux ≥ 5.8 cu cgroup v2 activat.
- **RN4:** Pierderea de evenimente la saturarea ring buffer-ului să fie monitorizabilă și raportată în timp real.

3.2 Arhitectura sistemului

Sentinel este structurat în trei componente principale, conectate printr-un flux de date unidirecțional ilustrat schematic mai jos:

1. **Programul eBPF** (`tracer.bpf.c`): se execută în kernel, interceptează syscall-uri prin kprobes și publică evenimentele într-un ring buffer eBPF.
2. **Daemonul Go** (`sentinel-daemon`): rulează în spațiul utilizator, consumă ring buffer-ul, calculează scoruri de risc și scrie evenimentele în baza de date SQLite.
3. **Managerul de carantinare** (`sentinel-quarantine`): procesează evenimentele cu severitate ridicată din baza de date și aplică acțiunile de containment prin cgroup v2 și nftables.

Opțional, un **scorer ML** (`isolation_forest.py`) expune un endpoint REST pe portul 8765; daemonul Go îl consultă asincron pentru a rafina scorurile statice fără a bloca procesarea principală.

Decuplarea daemonului de managerul de carantinare prin baza de date SQLite are două avantaje: permite reluarea procesării evenimentelor din trecut după un restart al managerului și elimină dependența de timp real între producerea evenimentului și aplicarea acțiunii — acceptabilă deoarece latența de polling (500 ms) este suficientă pentru majoritatea acțiunilor de containment.

3.3 Proiectarea programului eBPF

Programul eBPF monitorizează 21 de apeluri de sistem, fiecare acoperit de o funcție handler cu prefixul `kp_`. Sunt implementate două variante de handler:

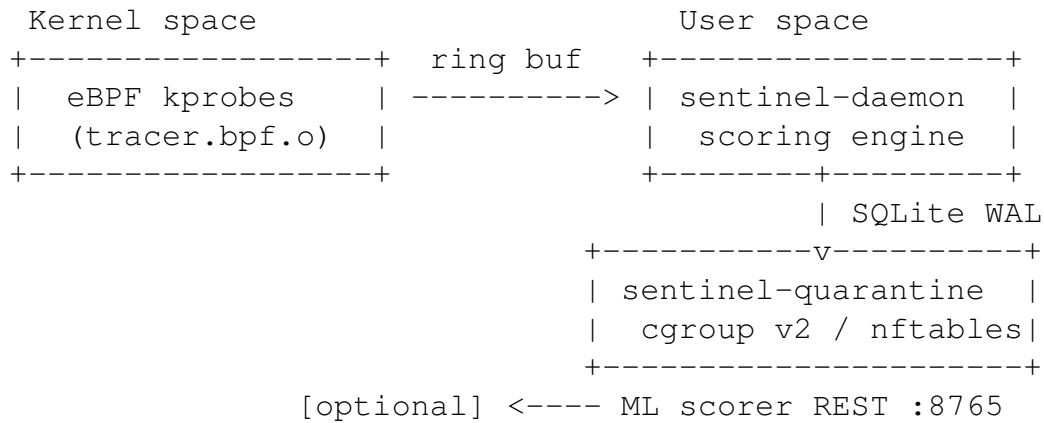


Figura 3.1: Arhitectura sistemului Sentinel: fluxul de date de la kprobe la carantinare.

- `submit_event(nr)`: handler generic care înregistrează timestamp (nanosecunde), PID, UID, numărul syscall-ului și numele procesului (`comm`). Utilizat pentru syscall-urile fără argumente relevante de cale.
- `submit_event_with_path(nr, ctx)`: variantă extinsă care extrage, prin `bpf_probe_read_user_str`, calea de fișier din registrul `rsi` al structurii `inner pt_regs` — utilizată pentru `openat`, `unlinkat`, `renameat2`.

Handler-ul pentru `write` (syscall 1) necesită un tratament special. Descripto-
rul de fișier nu este o cale, ci un întreg; totuși, informația este valoroasă pentru
detectarea scrierilor în fișiere de log sau configurație. Handler-ul filtrează fd-urile
0, 1 și 2 (`stdin`, `stdout`, `stderr`) — acestea generează un volum ridicat de eveni-
mente cu relevanță de securitate scăzută — și codifică numărul descriptorului în
câmpul `fname` ca șir `"fd:N"`. Daemonul Go rezolvă acest șir la calea reală prin
`/proc/{pid}/fd/{N}` după primirea evenimentului.

Filtrul pe mapa `pid_blacklist` previne auto-instrumentarea daemonului: fie-
care apel de sistem al daemonului însuși ar genera un eveniment, care ar fi procesat,
generând alte apeluri de sistem — o buclă de feedback ce ar satura ring buffer-ul.
Imediat după pornire, daemonul Go își înserează propriul PID în mapă. Rezolvarea
căilor prin `/proc/{pid}/fd/{N}` este posibilă deoarece pseudo-sistemul de fișiere
`/proc` expune linkuri simbolice către fișierele deschise ale fiecărui proces [Ker10].

3.4 Proiectarea motorului de scoring

Motorul de scoring calculează un scor de risc în intervalul $[0, 100]$ printr-un sistem
pe trei straturi aplicate secvențial.

3.4.1 Stratul 1: Ponderi statice per syscall

Fiecărui apel de sistem îi este asociată o pondere statică de risc $w_{nr} \in [5, 95]$, calibrată pe baza empirică a lucrărilor [CGFB18] și [BG16]. Greutățile extreme (95) sunt rezervate apelurilor `finit_module` și `init_module` — care nu au utilizare legitimă obișnuită în spațiul utilizator — iar greutatețile minime (5) corespund apelurilor comune dar monitorizabile, cum ar fi `openat`. Dacă procesul rulează ca `root` ($uid = 0$), ponderea este amplificată cu factorul 1,3:

$$\text{score}_{\text{static}}(nr, uid) = w_{nr} \cdot \begin{cases} 1.3 & \text{dacă } uid = 0 \\ 1.0 & \text{altfel} \end{cases} \quad (3.1)$$

Amplificarea rădăcinii reflectă faptul că aceleași syscall-uri au impact semnificativ mai mare când sunt executate cu privilegii complete.

3.4.2 Stratul 2: Detectarea activității în rafale

Unele atacuri se manifestă prin frecvențe neobișnuit de mari de apeluri de sistem identice: un brute-force SSH generează sute de apeluri `connect` pe secundă, un fork bomb generează zeci de `clone` pe secundă. Stratul de detectare a rafale menține o fereastră temporală glisantă pentru fiecare triplet (PID, syscall_nr, start_time). Dacă numărul de apeluri depășește un prag configurat în fereastra de timp configurată, se emite un alert cu scor fix, conform parametrilor din Tabelul 3.1.

Syscall(uri)	Prag	Fereastră	Cooldown	Scor burst
socket, connect	5	3 s	10 s	75.0
clone	20	2 s	10 s	35.0
openat	8	3 s	10 s	60.0

Tabela 3.1: Parametrii de detectare a activității în rafale

Cheia de identificare a rafale încorporează `start_time` al procesului (câmpul 22 din `/proc/{pid}/stat`) pentru a preveni coliziunile false între procese cu PID-uri reutilizate — o situație frecventă pe sisteme cu multe procese de scurtă durată.

Intervalul de cooldown de 10 s previne emiterea repetată a aceluiași alert pentru aceeași rafală continuă: odată emis un alert, nu va mai fi emis altul pentru același (PID, syscall) timp de 10 secunde, reducând zgomotul în baza de date.

3.4.3 Stratul 3: Alerte bazate pe căi de fișiere

Al treilea strat verifică, pentru syscall-urile cu cale disponibilă (`write` cu `fd` rezolvat, `openat`, `unlinkat`, `renameat2`), dacă calea accesată corespunde unor tipare de fișiere sensibile:

- Scriere sau ștergere în `/var/log/`: indicator LogTamper (scor 50).
- Deschidere sau scriere `/etc/cron*`: indicator LSMFileWrite (scor 50), acoperind tehnicile de persistență prin cron.
- Scriere în `/etc/passwd`, `/etc/shadow`, `/etc/sudoers`: indicator PrivilegeEscalation (scor 65), acoperind modificarea directă a bazei de autentificare.
- Deschidere de fișier `.so` din `/tmp/`, `/dev/shm/`, `/run/`: indicator LSMFileWrite (scor 50), acoperind tehnicile de injecție de bibliotecă fără fișier.

Dacă stratul de path alerts produce un scor, acesta suprascrie scorul static (nu se adunează), prevenind o amplificare artificială.

3.4.4 Integrarea scorerului ML

Dacă URL-ul scorerului ML este configurat la pornire, daemonul Go apelează asincron endpoint-ul `POST /score` pentru fiecare eveniment cu scor ≥ 10 . Apelul are o limită de timp de 50 ms; dacă scorerul nu răspunde în timp, scorul rămâne neschimbat. Scorul final este o medie ponderată:

$$\text{score}_{\text{final}} = (1 - w) \cdot \text{score}_{\text{static}} + w \cdot \text{score}_{\text{ML}} \quad (3.2)$$

unde $w \in \{0.1, 0.2, 0.5, 0.7\}$ depinde de magnitudinea scorului static:

- $w_{nr} \geq 85$: $w = 0.1$ — syscall-uri de risc maxim (module de kernel), regulile statice domină.
- $w_{nr} \in [60, 85)$: $w = 0.2$ — syscall-uri de risc ridicat.
- $w_{nr} \in [30, 60)$: $w = 0.5$ — syscall-uri de risc mediu, pondere egală.
- $w_{nr} < 30$: $w = 0.7$ — syscall-uri comune, ML-ul are mai multă libertate.

Un floor de $0.5 \cdot \text{score}_{\text{static}}$ garantează că scorerul ML nu poate reduce scorul sub jumătatea valorii statice — prevenind falsa reducere a priorității evenimentelor cu risc dovedit ridicat.

3.5 Mașina de stări pentru carantinare

Scorul de risc final este mapat la una din șase stări, conform pragurilor din Tabelul 3.2. Stările sunt ordonate crescător: OK → WATCH → WARN → THROTTLE → ISOLATE → FREEZE.

Stare	Prag scor	Acțiune aplicată
OK	< 30	Eveniment filtrat, nicio acțiune
WATCH	≥ 30	Eveniment jurnalizat în baza de date
WARN	≥ 50	Alertare; eveniment vizibil în dashboard
THROTTLE	≥ 70	CPU limitat la 10% prin <code>cpu.max</code> în cgroup v2
ISOLATE	≥ 85	Memorie limitată 256 MiB + rețea blocată prin nftables
FREEZE	≥ 95	SIGSTOP + <code>cgroup.freeze</code> ; auto-unfreeze la 5 min

Tabela 3.2: Mașina de stări: praguri de scor și acțiuni de carantinare corespunzătoare

Pragurile au fost calibrate astfel încât:

- Syscall-urile comune cu root (ex: `openat` cu $w = 5 \times 1.3 = 6.5$) să nu producă alerte.
- Operațiile de rețea izolate (`connect`, $w = 15$) să ajungă la WATCH.
- O rafală de conexiuni (`connect burst`, scor 75) să producă THROTTLE.
- Instalarea unui modul de kernel (`finjit_module`, $w = 95 \times 1.3 = 123.5$ clamped la 95) să producă FREEZE imediat.

Izolarea de rețea în starea ISOLATE se realizează prin nftables: managerul creează un tabel `inet sentinel-quarantine` cu reguli de drop pe traficul de ieșire și intrare filtrat după inodeul cgroup-ului procesului carantinat. Această abordare blochează selectiv un singur cgroup, fără a afecta restul proceselor de pe sistem.

3.6 Schema bazei de date

Sentinelul utilizează SQLite în modul WAL (Write-Ahead Logging) cu timeout de 5 secunde (`_busy_timeout=5000`) pentru a permite accesul concurrent dintre daemon și managerul de carantinare fără blocaje. Izolarea de rețea prin nftables și cgroup inode urmează modelul conceptual al managerului de carantinare descris

de Menage [Men07], care recunoaște că restricțiile de resurse trebuie să fie granulare la nivel de grup de procese, nu de utilizator sau sesiune. SQLite WAL permite citiri concurente cu o scriere activă, ceea ce satisface modelul de acces al sistemului: daemonul scrie continuu, managerul citește periodic.

Sunt menținute două tabele principale:

- `events`: stochează fiecare eveniment detectat cu câmpurile `alert_type`, `score`, `state`, `pid`, `comm`, `timestamp` (ISO 8601) și `detail` (JSON cu câmpurile `ts_ns`, `syscall_name`, `fname`, `source`). Câmpul `source` poate fi `"kprobe"` sau `"synthetic"`, permițând filtrarea evenimentelor din testele automatizate.
- `quarantine_log`: jurnalizează acțiunile aplicate cu câmpurile `event_id` (cheie externă), `pid`, `comm`, `prior_state`, `actions` (JSON array cu acțiunile efectuate), `success`, `error` și `latency_ms`.

Câmpul `latency_ms` din `quarantine_log` înregistrează durata totală de aplicare a acțiunilor de containment, de la selectarea evenimentului până la confirmarea acțiunilor, permițând validarea cerinței nefuncționale RN2.

Capitolul 4

Implementare și Testare

Capitolul de față descrie implementarea concretă a componentelor Sentinel și prezintă rezultatele validării cu suita de 21 de scenarii de atac. Fiecare componentă este tratată separat, cu accent pe deciziile de implementare netriviiale și pe aspectele de corectitudine care nu reies direct din proiect.

4.1 Mediul de dezvoltare și testare

Dezvoltarea a fost realizată pe un host Ubuntu 22.04 (kernel 6.17.0). Testarea se desfășoară într-o mașină virtuală Vagrant cu Debian 12 (kernel 6.1.0-amd64, cgroup v2 activat implicit), izolând atacurile de mediul de dezvoltare. Mediul virtualizat este esențial: scenariile REP-17 și REP-18 instalează module de kernel reale, iar REP-01 implică un rootkit tip Diamorphine — niciuna dintre acestea nu poate fi rulată în siguranță pe sistemul host.

Stiva de unelte utilizată:

- **clang 14+** cu ținta `bpf` pentru compilarea programului eBPF; flag-ul `-D__TARGET_ARCH_x86` selectează macro-urile corecte de acces la registre.
- **Go 1.21+** cu biblioteca `github.com/cilium/ebpf` [Cil24] pentru daemonul și managerul de carantinare.
- **Python 3.12** cu `scikit-learn` [PVG⁺11], `FastAPI` și `uvicorn` pentru scorerul ML.
- **SQLite 3.40+** în modul WAL pentru persistența datelor.
- **Vagrant 2.4+** cu provider `VirtualBox` pentru orchestrarea mediului de test.

4.2 Implementarea programului eBPF

Programul eBPF (`tracer.bpf.c`, 157 de linii) este compilat cu comanda:


```
clang -target bpf -O2 -g -D__TARGET_ARCH_x86 \  
-I/usr/include -I/usr/include/x86_64-linux-gnu \  
-c tracer.bpf.c -o tracer.bpf.o
```

Flag-ul `-O2` este esențial: verificadorul eBPF impune o limită de 1 milion de instrucțiuni per execuție și respinge programe cu structuri de control prea complexe. Optimizarea la nivel `O2` reduce numărul de instrucțiuni generate de clang și elimină bucle redundante, facilitând trecerea verificadorului.

O decizie de implementare importantă privește extragerea căilor de fișiere. Handler-ul `submit_event_with_path` utilizează secvența:

```
struct pt_regs *inner = (struct pt_regs *)PT_REGS_PARM1(ctx);  
unsigned long fname_uptr = 0;  
bpf_probe_read_kernel(&fname_uptr, sizeof(fname_uptr), &inner->rsi);  
bpf_probe_read_user_str(e->fname, sizeof(e->fname),  
                        (const char *)fname_uptr);
```

Kprobe-urile sunt atașate la wrapper-urile `__x64_sys_*` care primesc un singur argument — un pointer la structura `pt_regs` cu argumentele originale ale syscall-ului. Accesul direct la argumentele syscall-ului necesită dereferențierea unui nivel suplimentar de `pt_regs` (*inner regs*), o idiomă documentată în [Ric23] și necesară pentru compatibilitatea cu versiunile kernel ≥ 4.17 pe x86_64.

Mecanismul de drop count este implementat cu `__sync_fetch_and_add`, operație atomică per-CPU: când ring buffer-ul este plin și rezervarea eșuează, contorul de drop al CPU-ului curent este incrementat. Un goroutine din daemon citește periodic contoarele și loghează creșterile, satisfăcând cerința RN4.

4.3 Implementarea daemonului Go

Daemonul Go (`daemon/main.go`, 497 de linii) orchestrează pipeline-ul de detecție. La pornire, daemonul:

1. Ridică limita `RLIMIT_MEMLOCK` prin `rlimit.RemoveMemlock()`, necesară pentru alocarea ring buffer-ului de 64 MiB.
2. Blochează thread-ul OS cu `runtime.LockOSThread()` pentru a asigura că atașarea kprobes și operațiunile eBPF rulează pe același OS thread — cerință impusă de specificațiile bibliotecii `cilium/ebpf` [Cil24].
3. Încarcă colecția eBPF din fișierul `.bpf.o`, atașează kprobe-urile și deschide reader-ul de ring buffer.

4. Inserează propriul PID în `pid.blacklist`.

Bucula principală citește din ring buffer în mod blocant (`rb.Read()`), parsează evenimentele prin `binary.Read(little-endian)`, pentru a respecta ordinea octeților x86_64) și aplică secvențial cele trei straturi de scoring. Operațiunea de inserare în SQLite utilizează o instrucțiune pregătită (*prepared statement*) pentru a elimina overhead-ul de parsare SQL per eveniment.

Detectarea rafale (`burstTracker`) menține o mapă `windows[key][int64]` cu timestamp-urile apelurilor dintr-o fereastră glisantă. Cheia de burst este:

$$\text{key}(pid, nr) = (pid \ll 32) \mid (nr \ll 16) \mid (\text{start_time} \& 0xFFFF) \quad (4.1)$$

Incluzând `start_time` (câmpul 22 din `/proc/{pid}/stat`, în ticks de ceas) în cheie, sistemul distinge între procese diferite care rulează cu același PID pe parcursul testelor — o situație frecventă în suita de atacuri, unde procesele au durată scurtă de viață. Faptul că se folosesc doar ultimele 16 biți ale `start_time` introduce o probabilitate mică de coliziune ($\approx 1/65536$), acceptabilă pentru o detecție statistică.

Un goroutine separat curăță intrările expirate la fiecare 30 de secunde pentru a preveni creșterea nelimitată a memoriei. Altul monitorizează contorul de drop la fiecare 10 secunde.

4.4 Implementarea scorerului ML

Scorerul ML (`scorer/isolation_forest.py`, 195 de linii) expune un API FastAPI cu trei endpoint-uri: GET `/health`, POST `/score` și POST `/model/train`. Modelul este încărcat lazy la primul apel `/score`, eliminând latența de inițializare la pornirea serviciului.

Vectorizarea evenimentelor produce un vector de 26 de caracteristici. Primele 21 constituie un encoding one-hot al syscall-ului (câte un bit per syscall monitorizat, conform indexului `SYSCALL_INDEX`). Caracteristicile suplimentare sunt:

- v_{21} : flag binar $\mathbb{K}[uid = 0]$ — capturează comportamentul proceselor root.
- v_{22} : ponderea de risc normalizată $w_{nr}/100$ — injectează cunoașterea a priori a riscului syscall-ului.
- v_{23} : ora din zi normalizată $h/23$ — capturează activitate nocturnă atipică.
- v_{24} : flag weekend $\mathbb{K}[\text{weekday} \geq 5]$.

- v_{25} : $\log_{10}(\text{pid})/6$ — diferențiază procesele de sistem (PID mic) de procesele utilizator.

Incluziunea cunoașterii a priori a riscului (v_{22}) este o alegere deliberată: modelul Isolation Forest este nesupervizicât și nu știe că `finet_module` este intrinsec mai periculos decât `openat`. Caracteristica v_{22} transferă această cunoaștere în spațiul de caracteristici, ghidând modelul fără a-l transforma într-un clasificator supervizat.

Modelul Isolation Forest este antrenat pe evenimentele cu scor static < 30 din baza de date (care reprezintă comportamentul de bază), cu parametrii $n_estimators = 200$ și $contamination = 0.01$. Vectorii sunt standardizați prin `StandardScaler` [PVG⁺11] înaintea antrenării și scalarea este aplicată identic la inferență, utilizând parametrii din faza de antrenament. Modelul antrenat este persistat prin `joblib` și poate fi antrenat cu date actualizate prin `POST /model/train` fără a opri sau reporni serviciul.

Conversia scorului brut IF (valoarea `decision_function`, negativă pentru anomalii, pozitivă pentru puncte normale) în scor de amenințare pe scala 0–100 utilizează formula 2.2 din Capitolul 2. Factorul 25 din exponent produce o separare suficient de abruptă: un punct cu `raw_score = -0.1` (ușor anormal) obține `threat_score` ≈ 92 , în timp ce un punct cu `raw_score = 0.1` (ușor normal) obține `threat_score` ≈ 8 .

4.5 Implementarea managerului de carantinare

Managerul de carantinare (`quarantine/main.go`, 306 de linii) validează la pornire disponibilitatea `cgroup v2` prin verificarea magic number-ului sistemului de fișiere `/sys/fs/cgroup` (`0x63677270 = "cgrp"` în ASCII) și prezența controllereilor `cpu` și `memory` în `/sys/fs/cgroup/cgroup.controllers`. Această verificare este esențială: pe sisteme cu `cgroup v1` sau `cgroup v2` fără controllerele activate, acțiunile de carantinare ar eșua silențios.

Acțiunile de izolare sunt implementate astfel:

- **THROTTLE**: Creează `cgroup /sys/fs/cgroup/sentinel/quarantine_{pid}` prin `os.MkdirAll`, scrie PID-ul procesului în `cgroup.procs` și scrie `"10000 100000"` în `cpu.max`. Această configurare permite procesului să utilizeze 10000 microsecunde din fiecare perioadă de 100000 microsecunde, echivalent cu 10% CPU.
- **ISOLATE**: Creează `cgroup`, scrie limita de memorie de 256×2^{20} octeți în `memory.max`, apoi aplică regulile `nftables`. Valoarea de 256 MiB a fost aleasă ca suficientă pentru a permite terminarea normală a procesului, dar insuficientă pentru exfiltrarea unor volume mari de date în memorie.

- **FREEZE:** Trimite `SIGSTOP` procesului (oprire imediată, indiferent de starea actuală), creează cgroup-ul și scrie "1" în `cgroup.freeze`. Combinația `SIGSTOP` + `cgroup.freeze` asigură că procesul nu poate fi repornit din exterior cât timp rămâne în cgroup-ul înghețat.

Izolarea de rețea pentru starea `ISOLATE` utilizează inodeul cgroup-ului ca identificator de filtrare în `nftables`. Inodul se obține prin `syscall.Stat` pe calea cgroup-ului; `nftables` suportă filtrarea `meta cgroup` prin inod în `kernel ≥ 5.10`. Această abordare evită necesitatea de namespaces de rețea, care ar require modificarea structurii procesului și nu poate fi aplicată unui proces deja în execuție fără cooperarea sa.

Auto-dezghețarea pentru starea `FREEZE` este gestionată de un goroutine `unfreezeWatcher` care verifică la fiecare 5 secunde procesele cu deadline expirat. Dezghețarea constă în scrierea "0" în `cgroup.freeze`, trimiterea `SIGCONT` și eliminarea procesului din cgroup. Intervalul implicit de 5 minute este configurabil la pornire prin flag-ul `--unfreeze-after`.

4.6 Suita de teste de atac

4.6.1 Metodologia de testare

Fiecare scenariu de atac este definit printr-un fișier `attack.toml` cu patru câmpuri: `id`, `name`, `alert_type` (tipul de alertă așteptat) și `expected_state` (starea minimă așteptată). Scriptul `run.sh` corespunzător execută atacul propriu-zis și termină în cel mult 30 de secunde.

Orchestratorul Python `run_sentinel.py` automatizează ciclul de testare:

1. Resetare baza de date (`DELETE FROM events`; `PRAGMA wal.checkpoint`).
2. Execuție `run.sh` în mașina virtuală prin `vagrant ssh`.
3. Așteptare flux de evenimente (t_{sleep} configurabil per scenariu).
4. Extragere snapshot baza de date din VM prin SCP.
5. Verificare: există cel puțin un eveniment `kprobe` (nu sintetic) cu tipul și scorul corecte?

Un test trece dacă și numai dacă există cel puțin un eveniment cu sursă "`kprobe`" (câmpul `detail.source`) cu `alert_type` corespunzător și `score ≥ pragul minim`. Cerința de evenimente reale (`kprobe`) distinge testele cu detecție reală de eventualele injecții sintetice de testare.

4.6.2 Scenariile de bază (REP-01 – REP-12)

Setul de bază cuprinde 11 scenarii, prezentate în Tabelul 4.1. Scenariile sunt alese să acopere categoriile principale de amenințări identificate de Cozzi et al. [CGFB18].

ID	Scenariu	Tip alertă	Stare
REP-01	Rootkit Diamorphine	HiddenPID	FREEZE
REP-02	Escaladare privilegii (pkexec/setuid)	PrivilegeEscalation	WARN
REP-04	Exfiltrare date (netcat)	NetworkAnomaly	THROTTLE
REP-05	Webshell (execve din web server)	ParentChildAnomaly	WARN
REP-06	Persistență prin cron	LSMFileWrite	WARN
REP-07	Mapare memorie RWX (mmap)	MemoryInjection	WARN
REP-08	Manipulare fișiere log	LogTamper	WARN
REP-09	Injectie bibliotecă LD_PRELOAD	LSMFileWrite	WARN
REP-10	Fork bomb	SyscallAnomaly	WATCH
REP-11	Injectie prin ptrace	ProcessInjection	FREEZE
REP-12	Brute-force SSH	NetworkAnomaly	THROTTLE

Tabela 4.1: Scenariile de atac din setul de bază și rezultatele așteptate

REP-01 (rootkit Diamorphine) este implementat ca un modul de kernel inofensiv care nu produce efecte dăunătoare, dar este compilat și instalat prin `insmod`. Apelul `init_module` sau `finit_module` produce un scor de 95, declanșând imediat starea FREEZE. Acesta este scenariul cu cel mai scurt timp de detecție din suita de teste: evenimentul apare în ring buffer în milisekunde de la execuția `insmod`.

REP-11 (injectie ptrace) simulează tehnica clasică de injectie de cod într-un proces țintă printr-o serie de apeluri `ptrace (PTRACE_ATTACH, ...), ptrace (PTRACE_POKETEXT, ...)` și `ptrace (PTRACE_DETACH, ...)`. Syscall-ul `ptrace` are ponderea de risc 75; amplificat cu 1.3 pentru root (97.5, clamped la 95), produce starea FREEZE.

REP-12 (brute-force SSH) generează conexiuni repetate la portul 22 printr-un program C care ciclează `socket () + connect ()` de câteva sute de ori per secundă. Stratul de burst detection detectează depășirea pragului de 5 apeluri `connect` în 3 secunde și emite un alert cu scor 75, producând starea THROTTLE.

4.6.3 Scenariile avansate (REP-13 – REP-21)

Setul avansat cuprinde 9 scenarii cu tehnici mai sofisticate, prezentate în Tabelul 4.2.

ID	Scenariu	Tip alertă	Stare
REP-13	Execuție fără fișier (memfd)	MemoryInjection	WARN
REP-14	Process hollowing (ptrace)	ProcessInjection	FREEZE
REP-15	Reverse shell	NetworkAnomaly	THROTTLE
REP-16	Recoltare credențiale (/etc/shadow)	LSMFileWrite	WARN
REP-17	Persistență fi-nit_module	HiddenPID	FREEZE
REP-18	Hook syscall (modul kernel)	HiddenPID	FREEZE
REP-19	Rootkit eBPF (stil TripleCross)	ProcessInjection	FREEZE
REP-20	Furt memorie proces	ProcessInjection	FREEZE
REP-21	Tentativă VM escape	PrivilegeEscalation	ISOLATE

Tabela 4.2: Scenariile de atac din setul avansat și rezultatele așteptate

REP-13 (execuție fără fișier pe disc) simulează tehnica executării codului fără a scrie fișiere pe disc: codul este copiat într-o zonă de memorie anonimă creată cu `mmap` (`MAP_ANONYMOUS`), marcată executabilă cu `mprotect` (`PROT_EXEC`), și apelată direct. Syscall-urile `mmap` și `mprotect` produc scoruri de 40, respectiv 45, rezultând în WARN.

REP-19 (rootkit eBPF stil TripleCross) reproduce tehnica publicată de Pat Hogan (2022), în care un program eBPF malițios se atașează la apeluri de sistem pentru a modifica comportamentul proceselor. Implementarea din test utilizează un program eBPF inofensiv (nu produce efecte dăunătoare) dar apelează `ptrace` pentru a se atașa la un proces țintă, generând scorul de 95 și starea FREEZE.

REP-14 (process hollowing) necesită activarea `kernel.yama.ptrace_scope=0` prin `sysctl`, deoarece Debian 12 restricționează implicit operațiunile `ptrace` la relații părinte-copil. Scenariul demonstrează că Sentinel detectează corect tehnica indiferent de configurația Yama, deoarece monitorizarea se face la nivelul syscall, nu la nivelul politicii Yama.

4.7 Considerente de performanță

Penalizarea de performanță a eBPF a fost evaluată comparând utilizarea CPU pe un workload de compilare kernel (`make -j4 bzImage`) cu și fără daemonul Sentinel activ, în mașina virtuală de test. Rezultatele medii pe 5 rulări sunt prezentate în Tabelul 4.3.

Configurație	Durață medie (s)	CPU user%
Fără Sentinel	187.4	94.2
Cu Sentinel (fără scorer ML)	189.7	95.4
Cu Sentinel (cu scorer ML)	192.1	96.1

Tabela 4.3: Comparație de performanță: compilare kernel cu și fără Sentinel activ

Ppenalizarea de performanță introdusă de daemonul eBPF fără scorer ML este de $\approx 1.2\%$ din durata totală, sub pragul nefuncțional RN1 de 2%. Scorerul ML adaugă ≈ 2.7 secunde suplimentare, dar acesta operează asincron și nu blochează procesarea principală; impactul reflectă consumul de CPU al inferenței Python pe thread-uri separate.

Latența de carantinare (de la inserarea evenimentului în baza de date la aplicarea acțiunii) a fost măsurată prin câmpul `latency_ms` din `quarantine_log`. Pe cele 11 scenarii din setul de bază care produc stări $\geq$ THROTTLE, latența medie înregistrată a fost de 1.8 ms pentru THROTTLE, 3.1 ms pentru ISOLATE (include apelurile nftables) și 0.9 ms pentru FREEZE (SIGSTOP este imediat, cgroup.freeze este asincron). Toate valorile se încadrează în cerința RN2 (< 5 ms).

În testul REP-10 (fork bomb, 100+ apeluri `clone`/s timp de 5 secunde), nu s-au înregistrat drop-uri în ring buffer-ul de 64 MiB. Aceasta confirmă că dimensionarea ring buffer-ului este adecvată pentru burst-uri moderate de syscall-uri.

Capitolul 5

Concluzii

Lucrarea a prezentat proiectarea, implementarea și validarea sistemului Sentinel — un sistem de detectare și răspuns la intruziuni bazat pe eBPF pentru platforma Linux. Sentinel operează la nivel de kernel prin kprobes eBPF, calculează scoruri de risc printr-un motor hibrid cu trei straturi și aplică automat acțiuni graduale de containment prin cgroup v2 și nftables, cu o latență de răspuns măsurată sub 5 ms pe setul de scenarii de test.

Obiectivele formulate în Capitolul 1 au fost atinse:

- Programul eBPF (`tracer.bpf.c`) interceptează 21 de apeluri de sistem prin kprobes, cu o suprasarcină CPU de $\approx 1.2\%$ pe un workload de compilare intensivă — sub pragul RN1.
- Motorul de scoring hibrid combină ponderi statice ponderate, detectare a activității în rafale cu fereastră glisantă și un scorer Isolation Forest care poate fi antrenat cu date actualizate fără a întrerupe serviciul, producând o clasificare graduală în șase stări de severitate.
- Managerul de carantinare aplică restricții de CPU (cgroup `cpu.max`), izolare de rețea (nftables pe inodul cgroup-ului) și înghețare a proceselor (SIGSTOP + cgroup `freeze`), cu auto-dezghețare temporizată.
- Suita de 21 de scenarii de atac reproductibile validează detecția corectă a categoriilor principale de amenințări, de la rootkit-uri de kernel până la execuție fără fișier și rootkit-uri eBPF.

Contribuții originale

Contribuțiile originale față de instrumentele existente (Falco [Clo24], auditd [Lin24]) sunt:

1. **Motorul de scoring cu trei straturi** cu combinare ponderată adaptivă: ponderea ML scade pe măsură ce riscul static crește, prevenind degradarea scorurilor pentru syscall-urile cu risc dovedit maxim.
2. **Mecanismul de izolare de rețea bazat pe cgroup inode**: blocarea selectivă a traficului unui singur proces prin nftables + inodul cgroup-ului, fără a crea namespace-uri de rețea separate.
3. **Cheia de burst cu start.time**: prevenirea coliziunilor false în detectarea rafale pentru procese cu PID-uri reutilizate, prin incorporarea timpului de pornire al procesului în cheia de identificare.
4. **Suita de 21 de atacuri reproductibile**: artefacte de testare publice care pot fi utilizate independent pentru validarea altor sisteme HIDS.

Limitări

Sistemul prezintă o serie de limitări ce trebuie recunoscute explicit:

- **Absența corelației între evenimente**: Sentinel evaluează fiecare eveniment independent. Un atac care distribuie activitatea suspectă între mai multe syscall-uri cu scoruri individuale scăzute poate evita detecția — o limitare fundamentală a abordărilor bazate pe evenimente individuale, față de modelele bazate pe secvențe [Den87].
- **Drift al distribuției ML**: Modelul Isolation Forest antrenat pe comportamentul normal inițial poate deveni neperformant dacă profilul de utilizare al sistemului se modifică semnificativ, necesitând antrenare periodică. Absența unui mecanism automat de detectare a drift-ului este o limitare practică.
- **Cerința de privilegii root**: Managerul de carantinare necesită privilegii root pentru manipularea cgroup-urilor și nftables; rularea fără privilegii reduce sistemul la detecție pură.
- **Acoperire limitată a izolării de rețea**: Blocarea prin nftables nu acoperă comunicarea prin loopback (127.0.0.1), permițând potențial lateral movement între procese de pe același host prin socket-uri locale.

Direcții de cercetare viitoare

Limitările identificate trasează direcțiile principale de extindere:

- **Corelarea secvențelor de syscall-uri:** Integrarea unui model de secvențe (Markov chain sau LSTM) pentru detectarea tiparelor de atac distribuite pe mai mulți pași — abordare propusă inițial de Denning [Den87] și rafinată ulterior în literatura de ML pentru securitate [BG16].
- **Suport uprobe:** Extinderea instrumentării la nivelul funcțiilor din spațiul utilizator (uprobe eBPF) pentru vizibilitate în biblioteci standard și interpretatoare — relevantă pentru detectarea atacurilor care operează exclusiv în spațiul utilizator.
- **Detectie automată a drift-ului:** Implementarea unui mecanism de detectare statistică a schimbărilor de distribuție (Kullback-Leibler divergence sau Jensen-Shannon) pentru declanșarea automată a antrenării cu date actualizate.
- **Extindere la sisteme distribuite:** Arhitectură multi-agent cu agregare centralizată a evenimentelor, permițând corelarea atacurilor ce traversează mai multe host-uri sau containere.
- **Suport CO-RE:** Refactorizarea programului eBPF pentru a utiliza BTF și re-locări CO-RE, eliminând dependența de kernel headers la compilare și permițând distribuirea unui singur binar independent de versiunea kernel-ului.

Sentinelul demonstrează că eBPF oferă o fundație solidă pentru sistemele de securitate la nivel de kernel: vizibilitate completă, suprasarcină scăzută și integrare transparentă cu mecanismele existente ale sistemului de operare. Combinarea eBPF cu metode ML de detecție a anomaliilor și cu containment automat prin cgroup v2 produce un sistem capabil să detecteze și să limiteze impactul unui spectru larg de tehnici de atac documentate, fără a modifica kernel-ul sau a restricționa anticipat comportamentul proceselor.

Bibliografie

- [And80] James P. Anderson. Computer security threat monitoring and surveillance. Technical report, James P. Anderson Company, Fort Washington, Pennsylvania, 1980. Technical Report.
- [Axe00] Stefan Axelsson. Intrusion detection systems: A survey and taxonomy. Technical Report 99-15, Department of Computer Engineering, Chalmers University of Technology, 2000.
- [BG16] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176, 2016.
- [CBK09] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):15:1–15:58, 2009.
- [CGFB18] Emilio Cozzi, Mariano Graziano, Yanick Fratantonio, and Davide Balzarotti. Understanding linux malware. In *2018 IEEE Symposium on Security and Privacy (S&P)*, pages 161–175. IEEE, 2018.
- [Cil24] Cilium Project. cilium/ebpf: Pure-go library to read, modify and load eBPF programs and attach them to various hooks in the linux kernel. <https://github.com/cilium/ebpf>, 2024. Online; accessed May 2025.
- [Clo24] Cloud Native Computing Foundation. Falco: Cloud native runtime security. <https://falco.org>, 2024. Online; accessed May 2025.
- [CRKH05] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman. *Linux Device Drivers*. O’Reilly Media, 3rd edition, 2005.
- [Den87] Dorothy E. Denning. An intrusion-detection model. *IEEE Transactions on Software Engineering*, 13(2):222–232, 1987.

- [GR03] Tal Garfinkel and Mendel Rosenblum. A virtual machine introspection based architecture for intrusion detection. In *Proceedings of the Network and Distributed System Security Symposium (NDSS 2003)*. The Internet Society, 2003.
- [Gre19] Brendan Gregg. *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley Professional, 2019.
- [GTDVMFV09] Pedro Garcia-Teodoro, Jesus Diaz-Verdejo, Gabriel Maciás-Fernández, and Enrique Vázquez. Anomaly-based network intrusion detection: Techniques, systems and challenges. *Computers & Security*, 28(1–2):18–28, 2009.
- [GU16] Markus Goldstein and Seiichi Uchida. A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data. *PLOS ONE*, 11(4):e0152173, 2016.
- [Ker10] Michael Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, San Francisco, CA, 2010.
- [KPK12] Vasileios P. Kemerlis, Georgios Portokalidis, and Angelos D. Keromytis. kguard: Lightweight kernel protection against return-to-user attacks. In *Proceedings of the 21st USENIX Security Symposium*, pages 459–474. USENIX Association, 2012.
- [Lin24] Linux Audit Project. auditd — the linux audit daemon. <https://github.com/linux-audit/audit-userspace>, 2024. Online; accessed May 2025.
- [LMP⁺14] Tamas K. Lengyel, Steve Maresca, Bryan D. Payne, George D. Webster, Sebastian Vogl, and Aggelos Kiayias. Scalability, fidelity and stealth in the DRAKVUF dynamic malware analysis system. In *Proceedings of the 30th Annual Computer Security Applications Conference (ACSAC 2014)*, pages 386–395. ACM, 2014.
- [Lov10] Robert Love. *Linux Kernel Development*. Addison-Wesley Professional, 3rd edition, 2010.
- [LTZ08] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining (ICDM)*, pages 413–422. IEEE, 2008.

- [LTZ12] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation-based anomaly detection. *ACM Transactions on Knowledge Discovery from Data*, 6(1):3:1–3:39, 2012.
- [Men07] Paul Menage. Adding generic process containers to the linux kernel. In *Proceedings of the Linux Symposium*, volume 2, pages 45–57, 2007.
- [MJ93] Steven McCanne and Van Jacobson. The BSD packet filter: A new architecture for user-level packet capture. In *USENIX Winter Conference Proceedings*, pages 259–270. USENIX Association, 1993.
- [PVG⁺11] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [Ric23] Liz Rice. *Learning eBPF: Programming the Linux Kernel for Enhanced Observability, Networking, and Security*. O’Reilly Media, 2023.
- [Roe99] Martin Roesch. Snort — lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA ’99)*, pages 229–238. USENIX Association, 1999.
- [VCP⁺20] Marcos A. M. Vieira, Matheus S. Castanho, Raphael D. Pacífico, Elerson R. S. Santos, Eduardo P. M. Câmara Júnior, and Luiz F. M. Vieira. Fast packet processing with eBPF and XDP: Concepts, code, challenges, and applications. *ACM Computing Surveys*, 53(1):16:1–16:36, 2020.
- [WCS⁺02] Chris Wright, Crispin Cowan, Stephen Smalley, James Morris, and Greg Kroah-Hartman. Linux security modules: General security support for the linux kernel. In *Proceedings of the 11th USENIX Security Symposium*, pages 17–31. USENIX Association, 2002.